

AIS Glass POC — What We Built, What It Produces, and On What Basis

Oct 6, 2026 · @vijji babu

A complete account of the proof of concept for Asahi India Glass: the problem it addresses, how it works, the results it produces, the evidence behind those results, and what it does not yet prove.

The problem

Asahi India Glass sells automotive glass through 53 branches. Each branch decides how much of each product to hold. Get it wrong one way and a customer wants a windscreen you do not have; get it wrong the other way and cash sits on a shelf.

Their own data shows the scale of it. Over the 70 weeks we examined, **customers ordered 2,602,392 units and 2,133,961 were despatched** — a fill rate of **82%**. Put differently, **468,431 units of demand went unmet**, worth about **₹142Cr** at list price.

That gap is the question. Some of it is unavoidable; some is a product sitting at the wrong branch, or an order placed too late. Today those decisions rest on experience and spreadsheets.

This POC asks whether demand can be forecast accurately enough, per branch and per product, to put a number behind those decisions — and whether that number can be trusted.

The data they gave us

Five files, delivered 8 September 2026. We have confirmed all five are **byte-for-byte unchanged** from what they sent — nothing was edited to make anything work.

File	Size	What it gives us
Orders & Receipts	67.6 MB	775,912 order lines: what was asked for, what shipped, and when. The demand signal.
Sales Data FY 24–26	82.4 MB	Invoiced sales, used as a stand-in for demand before the order book begins
Stock in Hand	7.0 MB	One snapshot of stock, dated 1 August 2026
Location Master	0.02 MB	53 branches: region, lead time, truck size

File	Size	What it gives us
Substitution Mapping	0.2 MB	Which products can stand in for which

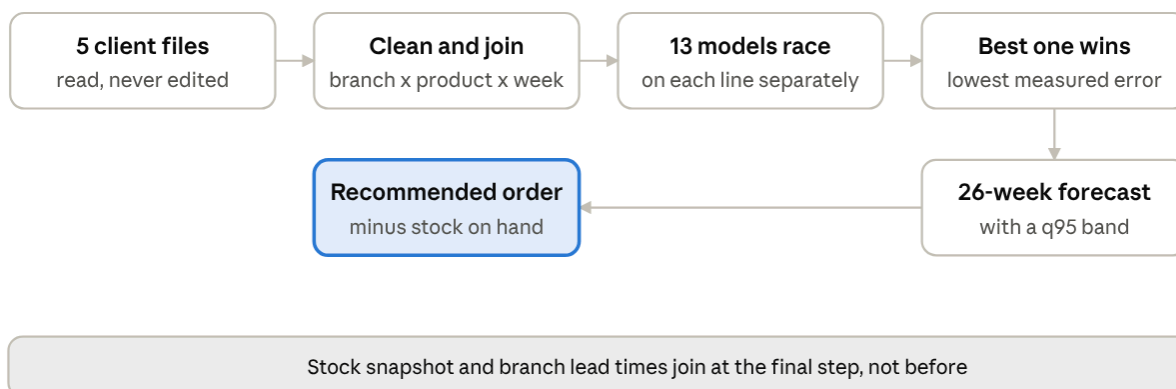
The order file is the important one. It records both what the customer asked for and what was actually sent, which is what makes the unmet-demand gap measurable at all. Most companies only keep what they sold.

Two limits in the data shape everything that follows. **The order book only starts in April 2025**, so earlier periods use invoiced sales as a labelled substitute — never mixed with real orders. And **stock is a single snapshot**, not a history, so nothing here can be tested against how stock actually moved.

What we built

From five client files to a recommended order quantity

Each step is measured and recorded; nothing is estimated between them



the pipeline · 6 steps from raw files to an order quantity

Step 1 — read the files. Nothing is written back to them, ever.

Step 2 — clean and join. Branch names and product codes are spelled inconsistently across files, so both are standardised. Orders are then grouped into one row per branch × product × week. Where the order book has not started yet, invoiced sales fill in, labelled as a substitute rather than silently mixed.

Step 3 — race 13 models. For every branch-and-product line, thirteen forecasting models are each fitted and scored. Not one model for everything: each line gets its own contest.

Step 4 — the best one wins. The model with the lowest measured error on that line becomes its champion. Across the current run the winners are genuinely spread — Exponential Smoothing takes 55 lines, LSTM 40, XGBoost 33 — which is the evidence that no single model would have done.

Step 5 — forecast 26 weeks. Each champion forecasts its own line, with a q95 band: a level that covers demand 95 times out of 100.

Step 6 — turn it into an order. The forecast is scaled to the period that needs covering (how long until the next order, plus the branch's delivery time), then stock on hand is subtracted.

Alongside this, four deliberately simple rules — "repeat last week", "repeat last year", 3-week and 6-week averages — run as a control. They can never win, but if a real model cannot beat them, that is reported rather than hidden.

What it produces

Forecast accuracy depends entirely on how far ahead you are planning, and that is the most important result in this document.

Measured on 261 branch × product lines, against demand the models had never seen:

Planning window	Accuracy	Typical error	Lines hitting the 85% target
One month at a time	71.45%	28.6%	28 of 261
A 3-month total	82.95%	17.1%	104 of 261
A 6-month total	89.55%	10.5%	159 of 261

A single week or a single month is genuinely hard to call — glass demand at one branch for one product is lumpy. But **the six-month total, which is what a replenishment decision actually rests on, lands within about 10%.** That clears the 85% target the project set itself.

249 of 261 lines get more accurate as the window lengthens; only 10 get worse. Errors in individual weeks cancel out rather than compound, which is the behaviour you want when the question is "how much should I hold" rather than "what will sell next Tuesday".

What else it produces

- **A recommended order quantity** for every branch × product line, sized to cover the review period plus the delivery lead time, at a 95% service level.
- **A ranked action list** — currently 261 lines: 85 critical, 155 high, 11 medium, 10 where a figure could not be produced and the page says why.
- **A demand picture across the whole network** — all 53 branches and 2,063 products, independent of what was modelled.

- **A lead-time check** comparing the branch master's stated lead time against what the order dates actually show.

One finding worth the client's attention

40 of the 272 modelled lines are beaten by a trivial rule — "repeat last period" or a simple average. We report that rather than hiding it. On those lines a model is not earning its complexity, and that is useful information about which products are genuinely forecastable.

On what basis

An accuracy figure is only worth as much as the way it was measured. Here is exactly how.

The models are tested on data they never saw. We cut the history at a point in the past, train only on what came before it, then ask for a forecast and compare against what actually happened next. The test data is never part of the fitting. Everything — the scaling, the feature building, the parameter tuning — happens inside the training window only.

The split is always chronological, never random. A random split would let the model learn from next March while predicting last March. For a time series that inflates accuracy and means nothing in production.

Thirteen models compete on every line, and the best one wins that line. Not one model applied everywhere. The winners are spread — Exponential Smoothing on 55 lines, LSTM on 40, XGBoost on 33, Auto ARIMA on 26, and so on. A product with steady demand and one with lumpy demand genuinely want different models.

Four deliberately trivial rules run alongside as a control — repeat last period, repeat last year, 3-period average, 6-period average. They can never win, but if a real model cannot beat "repeat last month", we say so.

A model that cannot run is shown, not hidden. It appears with a status — Ineligible, Failed, Timed out — and the exact reason. Dropping it would make the field look stronger than it is.

What the forecast is actually built on: each line's own demand history, plus a detected seasonal rhythm, plus the calendar. There are no external drivers — no vehicle registrations, no promotions, no pricing. Worth being plain about: the models say *this line has behaved like this and should continue*, not *this will happen because of that*.

How we know the numbers are right

We did not take our own pipeline on trust. We recomputed every headline figure **directly from the client's raw files** and compared.

Across the whole network — 53 branches, 2,063 products, 70 weeks

Figure	Their file	Our analysis	
Units ordered	2,602,392	2,602,392	Exact
Units despatched	2,133,961	2,133,961	Exact
Order value	₹857.10Cr	₹857.11Cr	0.001%
Short units	504,052	504,052	Exact
Fill rate	82.00%	82.00%	Exact
Branch totals	all 53	all 53	Exact

On the modelled slice — 2 branches, 136 products

Figure	Their file	Our analysis
Units ordered	146,814	146,814
Units despatched	129,559	129,559
Order value	₹59.95Cr	₹59.95Cr
Fill rate	88.25%	88.25%

Every chart also adds back to its own headline. Branch, value class, glass type, vehicle category, vehicle age, product group and the time trend each sum to 2,602,392 units exactly.

Three defects this check found, all fixed

1. **Order value overstated by ₹2.92Cr** — prices were averaged within each weekly bucket before being applied to the quantity. Now weighted by quantity.
2. **246 units wrongly counted as short** — a blank despatch quantity was read as zero despatched. Blank means *not recorded*, not *nothing shipped*.

3. **2,641 units missing from five charts** — lines whose product is absent from the client's catalogue were being dropped. They now appear under a labelled category so the bars add up.

None of these affected the forecasts: the models predict ordered quantity, which was correct throughout.

What is proven, and what is not

Worth having straight before anyone asks.

Proven

- The demand picture reconciles to the client's own files, exactly, at both network and pilot scope.
- Six-month demand can be forecast to about 10% error on 261 branch × product lines, measured on data the models never saw.
- The pipeline runs end to end, from raw files to a recommended order quantity, and is reproducible.

Not proven

- **That acting on the recommendations would have helped.** Testing that needs stock history, and the client's files contain only a single snapshot dated 1 August 2026. There is nothing to replay a policy against. This is the single biggest gap, and no figure in the POC should be read as a saving.
- **That the models generalise beyond the pilot.** 2 branches and 136 products — about 9% of network volume. Those 136 were chosen for being high value and steady; slow-moving products are harder, and we have not measured them.
- **Forecast quality at a single week or month.** 71% at one month against 90% at six. Short-horizon planning is not what this currently supports well.

Known limits in the source data

- The order book starts April 2025; earlier history is invoiced sales used as a stand-in, labelled differently throughout.
- 142,482 order lines were short-shipped, so the recorded order is a floor on true demand — the real figure is higher by an unknown amount.
- Two products ordered across most of the network are missing from the client's own catalogue.
- Lead time is measurable only to despatch; there is no goods-receipt date, so the final leg is invisible.

From here to production

Four things, roughly in order of what they would change.

1. Stock history — the one that unlocks everything. A monthly stock snapshot going back a year or two would let us replay the recommendations against what actually happened and say, with evidence, what the policy would have saved. Without it the POC can describe and forecast but never prove value. This is the ask to put to the client first.

2. Widen the modelling scope. The pipeline already handles the full network descriptively. Extending the forecasting from 136 products to a few thousand is compute, not new engineering — and it would show how the models hold up on slower-moving products, which is the honest test.

3. External demand drivers. Today the models use each line's own history plus the calendar. Vehicle registrations or parc data by region, or a promotions calendar, would plug straight into the existing structure and are the natural next lever on accuracy.

4. Operational fit. Open-order and backorder positions are currently assumed to be zero because the source files carry neither. A live feed would make the recommended quantities directly actionable rather than indicative.

Two small items for the client

- ☐ Supply the two missing product records — one is ordered by 44 of their 53 branches
- ☐ Confirm whether a blank despatch quantity means "not shipped" or "not recorded"; we have assumed the latter

Our view: the POC does what it set out to do. The demand picture is sound and reconciles to their data, and six-month forecasting works at the accuracy the project targeted. The gap between this and a business case is stock history, and that is a data request rather than a development task.